

Predictable Random Number Generation

This vulnerability arises when a smart contract tries to generate “random” values by using environmental data that is predictable, manipulable, or observable by adversaries—especially miners or validators.

Specification primitives Our defect specifications rely on standard Solidity datatypes— **Contract**, **Method**, **Address**, **Env**(ironment), and **State**. A brief description of the necessary predicates and functions over these types is included in Table 1. The evaluation of these primitives is stratified: structural properties are derived statically via Slither, while behavioral properties are verified at runtime via Forge/Kontrol assertions within the LLM-generated test harness.

Table 1. Primitives for Predictable Random Number Generation Specification

Symbol	Semantics
<i>Static Analysis (Slither)</i>	
<code>methods(c)</code>	The set of public/external methods defined in contract c .
<i>Runtime Verification (LLM-generated based on specific template)</i>	
<code>IsCriticalDecision(v)</code>	Holds if a value v is used as the primary seed for a state transition with high financial impact (e.g., determining a lottery winner or NFT rarity).

Domain Conceptual Model. This specification focuses on the purpose of the generated random value. It distinguishes between using environmental data like a timestamp for a simple expiration check (which is generally safe) versus using it as a seed for a critical decision, such as a lottery or reward distribution (which is highly vulnerable).

Formal Specification.

$$\forall m \in \text{methods}(c). \exists \mathcal{F} : \mathcal{E} \rightarrow \text{Result}. \exists \mathcal{E}_{unsafe} \subset \text{Env}. \exists v \in \text{Result}. \\ \text{Predictable}(c) \equiv v = \mathcal{F}(\mathcal{E}_{unsafe}) \wedge \text{IsCriticalDecision}(v)$$

where

$\mathcal{E}_{unsafe} \subset \text{Env}$ is the subset of the blockchain environment variables that are known to be predictable or manipulable by miners and validators. This includes properties like `block.timestamp`, `block.number`, and `block.prevrandao`.

Specific test template design.

Testing Goal Prove that a critical decision or value, intended to be random, is a deterministic function of the unsafe observable environment set.

Setup The harness utilizes Forge cheatcodes, such as `vm.warp` and `vm.roll`, to fix environmental variables like `block.timestamp` and `block.number` to specific fuzzed or symbolic values.

Correlation Check The test is designed to execute the target “random” function multiple times within the same simulated block or across identical environmental parameters to verify consistency.

Action In the test contract, the LLM instantiates a mirroring logic to pre-calculate the expected result using the same observable block data (e.g., `uint(keccak256(abi.encodePacked(block.timestamp)))`).

Assertion The framework asserts that the contract’s output matches the pre-calculated value (`actualResult == preCalculatedResult`). If these values match across multiple fuzzed environments, the randomness is confirmed to be predictable and potentially manipulable by miners or validators.

Template Derivation and LLM Instantiation The conceptual test design is materialized into a standardized Foundry test template. To bridge the gap between static analysis and generation, the template embeds explicit `[LLM_INSTRUCTION]` comments to guide the LLM through the instantiation phase. The process unfolds as follows:

Target Initialization (Setup): The LLM is instructed to import the target smart contract artifact and initialize the contract under test. It then analyzes the target’s source code to infer specific block dependencies (e.g., `block.timestamp`, `block.number`, `block.prevrandao`) to dynamically structure the fuzzing parameters.

Input Parameterization (Fuzzing/Setup): To support fuzzing, the LLM parameterizes the test function with the inferred block properties. It uses `vm.assume` to safely constrain these symbolic variables (preventing arithmetic overflows) and leverages Forge cheatcodes like `vm.warp` or `vm.roll` to explicitly manipulate and fix the simulated blockchain environment to those exact parameters.

Correlation Check: The LLM may optionally insert a correlation check, calling the target random function multiple times within the identical block state to ensure the output remains purely deterministic and does not differ internally.

Exploit Simulation (Action): The LLM synthesizes the action phase by replicating the vulnerable generation logic locally inside the test harness. Because the environment variables are explicitly controlled, the LLM can pre-calculate the exact “random” outcome the contract is expected to produce, storing it as `expectedRandom`. It then calls the contract to capture the `actualRandom` value.

Standardized Validation (Assertion): The test framework validates the defect by asserting `actualRandom == expectedRandom`. If the values reliably match across the fuzzed environments, the test decisively proves the randomness is fully predictable and determined by the observable block state.

```

1 // SPDX-License-Identifier: MIT
2 pragma solidity 0.8.29;
3
4 import {Test} from "../lib/forge-std/src/Test.sol";
5 // [LLM_INSTRUCTION]: Import the artifact of the contract being tested. The
   // solidity files are in "../src/". The name of the file is the same as the
   // name of the contract.
6 // [LLM_INSTRUCTION]: If needed, import specific cheats or libraries.
7
8 // -----
9 // [Testing Goal] Prove that a critical decision or value, intended to be
   // random,
10 // is a deterministic function of the unsafe observable environment set.
11 // -----
12
13 // [LLM_INSTRUCTION]: Name the contract 'TestWeakRandomness[ContractName]'
14 contract TestWeakRandomnessTemplate is Test {
15
16     // -----
17     // [Setup] Declare the contract under test variable.
18     // -----
19
20     // [LLM_INSTRUCTION]: Declare the contract under test variable
21     // BadRandomNumberGen public _contractUnderTest;
22
23     // ----- [Setup]
24
25     function setUp() public {
26         // -----
27         // [Setup] Initialize the contract under test.
28         // -----
29
30         // [LLM_INSTRUCTION]: Initialize the contract under test.
31         // 1. If constructor parameters are needed, use concrete valid values
32         // 2. If payable, use vm.deal(address(this), amount) before
           // deployment.
33
34         // _contractUnderTest = new BadRandomNumberGen();
35
36         // ----- [Setup]
37     }
38
39     // [LLM_INSTRUCTION]: INFER BLOCK DEPENDENCIES
40     // 1. Scan the contract source code for block properties.
41     // 2. Add arguments ONLY for the properties found:
42     // Found 'block.timestamp'? -> Add 'uint256 blockTimestamp'
43     // Found 'block.number'? -> Add 'uint256 blockNumber'
44     // Found 'block.prevrandao' or 'block.difficulty'? -> Add 'uint256
           // blockPrevrandao'
45     // Found 'block.coinbase'? -> Add 'address blockCoinbase'
46     //
47     // Example: function test_highlightPredictableRandomValue(uint256
           // blockTimestamp, uint256 blockNumber) public {

```

```

48     function test_highlightPredictableRandomValue(/* [LLM_INSTRUCTION]:
49         Insert inferred arguments here */) public {
50
51         // -----
52         // [Setup] Constrain the inferred symbolic variables and fix the
53         // environmental state using Forge cheatcodes (vm.warp, vm.roll, etc
54         // .)
55         // to specific fuzzed or symbolic values.
56         // -----
57         // [LLM_INSTRUCTION]: Constrain the inferred symbolic variables.
58         // Only generate lines for the arguments you added above.
59
60         // If blockTimestamp used:
61         // vm.assume(blockTimestamp > block.timestamp);
62
63         // If blockNumber used:
64         // vm.assume(blockNumber > block.number);
65
66         // [LLM_INSTRUCTION]: Avoid arithmetic overflows.
67         // Check the contract logic. If it sums these values, ensure they fit
68         // in uint256.
69         // Example: vm.assume(type(uint256).max - blockNumber >=
70         // blockTimestamp);
71
72         // [LLM_INSTRUCTION]: Funding (if needed)
73         // vm.deal(address(this), 100 ether);
74
75         // [LLM_INSTRUCTION]: MANIPULATE BLOCK ENVIRONMENT
76         // Apply the inferred values using Cheatcodes. Only generate lines
77         // for the arguments you added.
78
79         // If blockTimestamp used -> vm.warp(blockTimestamp);
80         // If blockNumber used -> vm.roll(blockNumber);
81         // If blockPrevrandao used -> vm.prevrandao(bytes32(blockPrevrandao))
82         // ;
83         // If blockCoinbase used -> vm.coinbase(blockCoinbase);
84
85         // ----- [/Setup]
86
87         // -----
88         // [Correlation Check] Execute the target "random" function multiple
89         // times within the same simulated block or across identical
90         // environmental parameters to verify consistency.
91         // -----
92
93         // [LLM_INSTRUCTION]: (Optional) Call the random function twice in
94         // the
95         // same block to check that the output is identical each time.
96         // uint256 firstCall = _contractUnderTest.getRandomNumber();
97         // uint256 secondCall = _contractUnderTest.getRandomNumber();
98         // assertEq(firstCall, secondCall, "Output differs within the same
99         // block - not purely block-dependent");
100
101         // ----- [/Correlation Check]
102
103         // -----
104         // [Action] Instantiate mirroring logic to pre-calculate the expected
105         // result using the same observable block data (e.g.,
106         // uint(keccak256(abi.encodePacked(block.timestamp)))).

```

```

100 // -----
101
102 // [LLM_INSTRUCTION]: PREDICT THE OUTCOME
103 // Replicate the vulnerable logic locally inside the test.
104 // Since we control the block state, we can calculate the exact
    result the contract should produce.
105
106 // Example Logic Replicated:
107 // uint256 expectedRandom = uint256(keccak256(abi.encodePacked(block.
    prevrandao, blockNumber, blockTimestamp)));
108
109 // [LLM_INSTRUCTION]: EXECUTE
110 // Call the contract method.
111 // uint256 actualRandom = _contractUnderTest.getRandomNumber();
112
113 // ----- [/Action]
114
115 // -----
116 // [Assertion] Assert that the contract's output matches the
117 // pre-calculated value (actualResult == preCalculatedResult). If
118 // these values match across multiple fuzzed environments, the
    randomness is
119 // confirmed to be predictable and potentially manipulable by miners
    or validators.
120 // -----
121
122 // [LLM_INSTRUCTION]: ASSERT PREDICTABILITY
123 // Prove that the contract's "random" value matches our calculated
    expectation.
124 // assertEq(actualRandom, expectedRandom, "Randomness should be
    predictable given block state");
125
126 // ----- [/Assertion]
127 }
128 }

```

Listing 1.1. Predictable Random Number Generation Multi-Shot test template